Anthropic

Clearcat.Net | **FIRST ATTEMPT PASS** | **WWW.CLEARCATNET.COM**

(CCDV-F)

Claude Certified Developer – Foundations

Be Anthropic Certified Now!

CCDV-F Dumps PDF

Latest Real Exam Q&A, FIRST ATTEMPT PASS



678 Q&A. pdf

Full Premium Material PDF – Verified by Experts

Follow us on: Facebook | Instagram | LinkedIn | reddit | Twitter | Quora | YouTube

Send us your request/inquiry at clearcat.net@gmail.com or connect us for Live Support any time for **any certification exam dumps pdf** Or for **most asked Interview Q&A PDFs** to ensure your success in first try!!

YouTube.com /CLEARCATNET

t.Me /CLEARCATNET

Get any exam latest real exam questions PDF Now-

Visit us - www.CLEARCATNET.com

Mail us- clearcat.net@gmail.com

Live Support- https://t.me/CLEARCATNET

We also build professional premium resume by experts to get shortlisted top of profiles



1. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. During a legacy payment module analysis, the coordinator first assigns one subagent to map database tables and another to trace API callers. Both return useful findings. The coordinator then invokes a planning subagent to propose a migration strategy, but the plan ignores the database constraints and caller list already discovered, recommending changes that would break known integrations.
What should you change to make this orchestration more reliable?
A. Replace the specialized subagents with one long-running generalist subagent that performs discovery and planning together.
B. Instruct the planning subagent to infer missing dependencies from file names when prior findings are unavailable.
C. Allow subagents to message each other directly so the planning subagent can request missing analysis details.
D. Have the coordinator maintain shared investigation state and include relevant prior findings in each subsequent subagent prompt.

Answer: D
Explanation:
Coordinator-owned state is central to reliable multi-agent orchestration. In a coordinator-subagent pattern, the coordinator decomposes work, collects results, and decides which findings must be supplied to later subagents so they can produce grounded outputs.
The underlying principle is that subagents operate in *isolated task contexts*. They should not be designed as if they can automatically see prior coordinator conversations or other subagents' outputs. The coordinator should maintain a structured investigation state, such as discovered tables, caller lists, constraints, open questions, and confidence notes, then include the relevant subset in each downstream prompt.
Letting subagents communicate directly weakens the hub-and-spoke architecture by hiding information flow from the coordinator. Replacing specialists with one generalist can create context bloat and reduce the value of parallel specialized analysis. Asking a planning subagent to infer missing dependencies from file names is especially risky because it substitutes guesses for evidence.
For more on agent orchestration and subagent patterns, see the Agent SDK documentation and the Claude Code Sub-agents documentation.

2. Scenario: Structured Data Extraction You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems. Your extraction QA pass reviews Claude's JSON outputs before downstream ingestion. Reviewers dismiss many findings because the QA prompt flags harmless differences: inferred date formats, optional fields absent from the source, and wording variations that do not change extracted values. The current prompt says, "Check the extraction for accuracy and report any problems.".
What change would most effectively improve precision?
A. Increase the validation sample size and ask reviewers to manually ignore findings that are not actionable.

B. Add instructions that Claude should be conservative and report only findings where it feels highly confident.
C. Rewrite the QA prompt to define reportable errors, acceptable variations, and skip conditions with concrete examples for each category.
D. Require the QA pass to flag every schema field that is null, even when the source document omits it.
Answer: C
Explanation:
Explicit criteria are the most effective way to improve precision when a prompt produces false positives. A prompt like *"check for accuracy"* leaves Claude to infer what counts as an issue, so it may flag harmless formatting differences, valid nulls, or stylistic variations as problems.
The stronger approach is to define reportable categories, acceptable variations, and skip conditions. For example, report a value that contradicts the source or a source-present required field that was omitted, but skip optional fields absent from the document and normalized formats that preserve meaning.
Instructions such as *"be conservative"* or *"only report high-confidence findings"* are weak substitutes because they do not create operational decision boundaries. Flagging all null fields is also counterproductive in extraction systems, since nullable fields are often necessary to avoid fabrication when source data is absent.
The underlying principle is that precision improves when the model receives concrete decision rules and examples rather than broad quality goals. Learn more about prompt specificity and examples in Prompt Engineering.

3. Scenario: Structured Data Extraction You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems. Your invoice extractor currently asks Claude to "return valid JSON" with fields for vendor, invoice_date, line_items, and total_amount. In production, a small but persistent share of responses include markdown fences, explanatory text, or malformed commas, causing downstream parsers to reject otherwise correct extractions.
What change would most effectively improve structured output reliability?
A. Keep requesting JSON only, then strip markdown fences and trailing prose with regular expressions before parsing.
B. Add stronger prompt wording that forbids explanations, examples, markdown, comments, and any non-JSON characters in every response.
C. Define an extraction tool with a JSON schema for target fields, then consume the structured tool_use input directly.
D. Use assistant prefill with an opening brace and stop sequences, then parse the generated text as JSON.
Answer: C
Explanation:
Structured output reliability is highest when the application uses Claude's tool use mechanism with a JSON schema representing the desired extraction fields. The model emits a tool_use block whose input conforms to the declared schema, so the application consumes structured arguments rather than scraping JSON from natural language text.

The underlying principle is to avoid treating machine-readable output as ordinary prose. Prompt-only instructions, assistant prefill, and stop sequences can improve formatting, but they still rely on text generation and parsing. For production extraction pipelines, tool use with schemas directly addresses JSON syntax failures such as markdown fences, extra commentary, and malformed delimiters.
Regex cleanup is especially brittle because it creates an ad hoc repair layer that can silently corrupt data or fail on new formatting variants. Stronger wording is also insufficient when downstream systems require consistent parseability rather than best-effort compliance. Learn more in the Tool Use documentation and the Claude API & SDK documentation.

4. Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. A user requests a report comparing how proposed AI copyright rules affect music licensing, model training data, and independent film production across the same jurisdictions and dates. Logs show the coordinator routes the entire request to one document analysis pass, then asks the report agent to write separate sections. The final report deeply covers licensing, barely addresses training data, and gives recommendations that conflict across sectors.
What workflow change would most effectively improve the result?
A. Split the request into distinct concern threads, investigate them in parallel with shared constraints, then synthesize one unified cross-sector report.
B. Process the concerns sequentially from highest commercial value to lowest, finalizing each report section before starting the next concern.
C. Add an instruction that the report agent should mention every requested sector at least once before delivering the final answer.
D. Ask every subagent to independently analyze the full user request, then concatenate their outputs into the final report document.
Answer: A
Explanation:
Correct workflow pattern: Compound requests should be decomposed into distinct concerns that can be investigated with focused attention, while shared constraints such as jurisdictions, dates, source requirements, and definitions are passed into each investigation. The coordinator can then synthesize the findings into a single report that reconciles interactions, conflicts, and dependencies across the concerns.
The underlying architectural principle is to separate *focused parallel investigation* from *unified synthesis*. A single broad pass risks attention dilution, while independent full-scope analyses create duplication and inconsistent assumptions. Sequentially finalizing sections can also produce incompatible conclusions because cross-concern relationships are not evaluated until too late.
The reporting-stage instruction is a common anti-pattern: it asks the final writer to cosmetically cover every topic without ensuring that the research workflow gathered enough evidence for each topic. Reliable multi-step workflows assign distinct items, preserve shared context, and require synthesis before final reporting.
Learn more about agent orchestration and tool-based workflows in the Agent SDK documentation and the Tool Use guide.

5. Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to backend systems through MCP tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate. Production logs show that lookup_order and process_refund both return the same failure text, "Operation failed." The agent retries declined refunds, tells customers to try again later when they provided invalid order IDs, and escalates temporary timeout cases that would likely succeed on retry. What change would best improve the agent's recovery decisions?

A. Route all refund and order lookup failures to escalate_to_human, avoiding autonomous recovery entirely after backend errors.

B. Return MCP tool errors with isError plus category, retryability, and safe messages distinguishing transient, validation, business, and permission failures.

C. Standardize every tool failure as a generic message, then ask Claude to infer recovery steps from conversation context.

D. Retry every failed backend tool call three times, then escalate unresolved cases without exposing error details to Claude.

Answer: B

Explanation:

Structured error responses let an agent make different decisions for different failure modes instead of treating every backend problem as the same event. In practice, an MCP tool should use an error signal such as isError and include metadata like errorCategory, isRetryable, and a human-readable explanation that is safe to show or summarize to the customer.

The underlying principle is that recovery logic depends on the nature of the failure: *transient* errors may be retried, *validation* errors usually require corrected input, *business* errors such as policy declines should be explained without retrying, and *permission* errors may require escalation or access handling. Generic failure text prevents Claude from selecting the right path and often causes wasted retries, premature escalation, or misleading customer responses.

Fixed retry counts are an anti-pattern when used as the primary response to all failures, because they ignore whether the error is actually retryable. Escalating every tool failure is also too conservative for an 80%+ first-contact resolution target, since many cases can be recovered locally with the right error details.

Learn more about MCP tool behavior in MCP Tools and Claude tool orchestration in Tool Use.

6. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. An engineer asks the agent to explain how the "user export" capability works in a legacy repository. The final answer confidently covers REST controllers and serializers, but misses scheduled exports, admin-triggered jobs, and CLI invocations. Logs show every subagent completed successfully; their prompts were "inspect export controller," "trace export API request," and "summarize export endpoint tests.".

What should you change first to improve coverage on similar broad codebase questions?

A. Require each subagent to read every file matching export-related terms before returning any findings to the coordinator.

B. Run a fixed pipeline that always invokes controller, database, CLI, worker, and test subagents for every codebase query.
C. Strengthen the synthesis subagent prompt to infer missing workflows from naming conventions and common framework patterns.
D. Revise coordinator planning to identify plausible entry points, then delegate distinct code areas to subagents before synthesis.
Answer: D
Explanation:
The root issue is the coordinator's task decomposition, not subagent execution. The subagents completed successfully, but all assigned prompts focused on the REST API path, so the final answer missed other relevant workflows such as scheduled jobs, admin actions, and CLI commands.
For broad codebase exploration, the coordinator should first reason about the possible surfaces where a capability may appear, such as controllers, jobs, commands, tests, database models, and integrations. It can then delegate distinct areas to specialized subagents and aggregate their findings into a complete explanation.
Asking synthesis to infer missing workflows is an anti-pattern because it encourages unsupported guesses. Having every subagent read every matching file creates context bloat and attention dilution, while a fixed pipeline wastes work and ignores the need for dynamic routing based on query complexity.
The underlying principle is that a coordinator-subagent architecture depends on the coordinator to preserve *coverage and routing discipline*. Learn more about agent orchestration patterns in the Agent SDK documentation.

7. Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. Your document-analysis subagent receives 45 reports and policy papers in a single pass before synthesis. The final outputs cite many sources but miss source-specific caveats, merge incompatible methodologies, and sometimes contradict earlier extracted facts.
What workflow change would most effectively improve reliability?
A. Analyze each source in a focused pass with structured findings, then run a separate cross-source integration pass before reporting.
B. Partition sources randomly across parallel subagents, concatenate their summaries, and have the report agent polish the combined narrative.
C. Set a maximum number of synthesis iterations and stop once the report includes citations from every source category.
D. Send all sources to the synthesis agent together and instruct it to be more careful with citations and contradictions.
Answer: A
Explanation:
Prompt chaining is appropriate when a workflow has predictable stages that benefit from focused attention. In this case, the reliable structure is to analyze each source or small source group first, producing structured findings with caveats and provenance, then run a separate cross-source integration pass before report generation.

The architectural principle is to avoid forcing one model call to juggle too many competing responsibilities at once. A focused local pass improves extraction quality, while the integration pass explicitly compares evidence, identifies contradictions, and prepares a coherent synthesis for the report agent.
Simply asking the synthesis agent to be more careful leaves the overloaded design unchanged. Random parallel partitioning can improve throughput, but concatenating summaries without an integration step risks fluent but inconsistent reporting. Using a fixed iteration limit or citation-count target is also unreliable because those signals do not measure whether the system handled methodological conflicts or caveats correctly.
For more background on breaking complex tasks into focused prompt sequences, see Prompt Engineering and Agent SDK.

8. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. A senior engineer reports that Claude Code consistently follows your team's codebase exploration notes, legacy module warnings, and boilerplate conventions. New engineers who clone the repository see generic behavior instead, and no repository files changed when the senior engineer originally added those notes.
What is the most effective way to make this guidance consistent for the team?
A. Create a slash command that reminds Claude to apply the conventions whenever developers remember to invoke it.
B. Paste the conventions into the first prompt of each new session instead of changing repository configuration files.
C. Move the shared conventions into a project-level CLAUDE.md file and commit it so every clone loads them.
D. Ask each developer to copy the senior engineer’s personal memory file into their home directory before using Claude Code.
Answer: C
Explanation:
Project-scoped configuration is the right fix when Claude Code behavior must be consistent for everyone working in the same repository. Guidance stored in a project-level CLAUDE.md, such as .claude/CLAUDE.md or a root CLAUDE.md, can be committed and distributed through normal version control workflows.
The underlying principle is *scope alignment*: personal preferences belong in user-level memory, while team standards and repository-specific context belong in project-level configuration. If guidance was added locally but no repository files changed, the likely issue is that it was stored in a personal location rather than a shared project file.
Copying personal memory files, relying on optional slash commands, or pasting conventions at session start all create process-dependent behavior and configuration drift. These approaches are fragile because they depend on each developer remembering the same manual steps.
Learn more about Claude Code memory hierarchy and shared project guidance in CLAUDE.md Configuration and the broader Claude Code Overview.

9. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. Your repository's root CLAUDE.md has grown to 1,200 lines covering testing, API conventions, migration rules, deployment checks, and review norms. Engineers complain that small standards changes cause frequent merge conflicts, and Claude sometimes appears distracted by unrelated guidance. The team wants shared, version-controlled instructions, but needs a more maintainable layout without relying on developers to remember extra steps.
What should you do?
A. Split the guidance into focused topic files under .claude/rules/, such as testing.md, api-conventions.md, and deployment.md.
B. Keep one root CLAUDE.md and add stronger section headings telling Claude to consult only relevant parts.
C. Move the shared standards into ~/.claude/CLAUDE.md so each engineer loads the same instructions outside repository files.
D. Create slash commands for each standards topic and ask engineers to invoke the right command before coding.
Answer: A
Explanation:
Modular Claude Code configuration is the right fit when a shared CLAUDE.md has become too large to maintain effectively. Splitting guidance into topic-specific files under .claude/rules/keeps standards version-controlled while reducing the operational burden of editing and reviewing one monolithic instruction file.
The underlying principle is to keep persistent project context *organized by concern*. Testing, API conventions, deployment rules, and review norms can evolve independently, making changes easier to review and reducing accidental conflicts between unrelated guidance areas.
Moving instructions into ~/.claude/CLAUDE.md breaks team sharing because that file is scoped to one user. Keeping everything in one root file with stronger headings preserves the monolith and relies on attention management rather than configuration structure. Slash commands are the wrong abstraction because they require manual invocation and are better suited to task-specific workflows, not baseline project standards.
Learn more about Claude Code memory hierarchy and modular configuration in CLAUDE.md Configuration and related Claude Code concepts in Claude Code Overview.

10. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. A coordinator agent delegates codebase exploration to subagents before asking an implementation subagent to generate migration scaffolding. In reviews, engineers find the final proposal often mixes findings from different packages, cites helper functions without file locations, and cannot explain which search result or source file supports a recommended change. The individual subagents found useful facts, but their handoffs were free-form summaries.
What change would best improve downstream reliability while preserving attribution?

A. Require subagents to return structured handoff records with findings separated from file paths, symbols, line ranges, commands, and source excerpts.
B. Strip source details from subagent outputs to reduce context size, then use Grep later when reviewers request justification.
C. Ask the implementation subagent to reread the repository broadly and infer supporting locations from each summarized recommendation before editing.
D. Have each exploration subagent write longer narrative summaries that include reasoning traces and repeated reminders to cite sources.
Answer: A
Explanation:
Structured subagent handoffs are the best fit when one agent's findings must be used by another agent for implementation or synthesis. The coordinator should require each exploration subagent to return records that distinguish the discovered fact from metadata such
as file_path, symbol, line_range, source_excerpt, and the command or tool result that produced the observation.
The underlying principle is that subagents operate with isolated context, so any downstream agent only receives what the coordinator explicitly passes along. Free-form summaries often compress away provenance, which makes later recommendations harder to verify and easier to misattribute across similar packages or duplicated helper functions.
Asking a downstream implementation agent to infer locations, adding longer narrative reasoning, or stripping metadata to save space all fail because they treat provenance as optional. In production developer tools, attribution is part of the work product, not decoration, because engineers need to validate recommendations against concrete files and source evidence.
Learn more about subagent orchestration in Agent SDK and Claude Code agent patterns in Claude Code Sub-agents.

11. Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. Over the weekend, several source documents were revised, a search index was refreshed, and two previously collected subagent outputs now cite superseded versions. The coordinator still has a saved investigation session containing the earlier web results, document analyses, and synthesis notes.
What is the most reliable way to continue the research without contaminating the final report with outdated evidence?
A. Create fork_session branches from the saved investigation and compare which branch produces the most internally consistent final report.
B. Start a fresh session with a structured summary of still-valid findings, then re-run targeted searches and analyses for changed sources.
C. Resume the saved session with --resume and instruct the coordinator to ignore any results produced before the weekend.
D. Resume the saved session and cap the coordinator to three additional iterations before generating the updated cited report.
Answer: B

Explanation:
Correct approach: When prior tool outputs are stale, the most reliable pattern is to start with a clean context and inject only a structured summary of information that remains valid. This preserves continuity without letting old search results, outdated document analyses, or superseded citations compete with refreshed evidence.
Underlying principle: Session resumption is useful when the prior context is mostly valid, but stale tool results can distort subsequent reasoning because they remain part of the conversation history. A structured summary acts as a controlled handoff: it carries forward durable decisions, open questions, and validated findings while requiring changed sources to be rechecked.
Why the alternatives fail: Simply resuming with --resume and telling the coordinator to ignore old results relies on prompt compliance while leaving invalid evidence in context. Using fork_session explores alternative approaches, but it branches from the same stale baseline. Adding an arbitrary iteration cap is an anti-pattern because it controls runtime rather than evidence quality or freshness.
Learn more about Claude Code session workflows in the Claude Code CLI documentation and broader agent patterns in the Agent SDK documentation.

12. Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. Your web search MCP tool occasionally times out. Today it returns a content block reading "Search provider timeout, try again later" as if the call succeeded. The synthesis subagent sometimes treats that text as a source note, and the report generator includes unsupported citations.
What tool response change best prevents the model from confusing failures with valid research content?
A. Convert failures into empty successful result lists, allowing the coordinator to proceed without additional error-handling complexity.
B. Keep returning error text as normal content, but instruct synthesis to ignore passages containing timeout or unavailable.
C. Return failed tool calls with isError set true, plus concise diagnostic content the subagent can use for recovery.
D. Raise uncaught server exceptions for all failures, letting the agent infer failure details from transport-level interruptions.
Answer: C
Explanation:
Correct approach: MCP tools should communicate failed executions as failures, not as ordinary content. Setting isError on the tool result tells the agent that the returned content is diagnostic information, not evidence to cite or synthesize.
Underlying principle: agents need machine-readable signals to separate valid empty results from tool failures. Once a timeout is marked as an error, the coordinator or subagent can retry, use an alternate source, or annotate a coverage gap instead of accidentally treating the message as research content.
Anti-patterns: Asking the model to infer errors from phrases like "timeout" or "unavailable" is brittle natural language parsing. Returning an empty successful list silently suppresses an access failure, while uncaught exceptions can terminate or obscure the workflow rather than enabling intelligent recovery.
For more details on MCP tool behavior and tool result design, see MCP Tools and Anthropic's Tool

Use documentation.

13. Scenario: Structured Data Extraction You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems. Your invoice extraction pipeline already returns syntactically valid structured objects, but semantic validators reject about 7% of documents because line-item totals do not sum to the stated total or normalized dates fail business rules. Most rejected documents contain the needed values, but the model placed or formatted them incorrectly.
What should you change to recover these cases while preserving validation rigor?
A. Ask Claude to explain likely extraction mistakes in natural language, then parse that explanation to update records.
B. Send a follow-up request containing the source document, failed extraction, and precise validator messages for targeted correction.
C. Retry the original extraction prompt up to three times, accepting the first response that passes schema validation.
D. Loosen the JSON schema by making rejected fields nullable, then route missing values to downstream reconciliation jobs.
Answer: B
Explanation:
Retry with targeted feedback is the right pattern when validation failures are caused by format, placement, or semantic consistency errors and the required information is present in the source. The retry should include the original document, the failed extraction, and the exact validation messages so Claude can focus on correcting the specific issue rather than redoing the task blindly.
The key tradeoff is preserving strict validation while enabling recoverable self-correction. JSON schema or tool-based structured output can reduce syntax problems, but it does not eliminate semantic errors such as totals that do not reconcile or values placed in the wrong field. Learn more about structured output patterns in Tool Use and prompt iteration in Prompt Engineering.
Blind retries with arbitrary caps are an anti-pattern because they consume additional calls without giving the model actionable information. Loosening schemas to bypass failed validation weakens data quality guarantees, and parsing natural language explanations creates a brittle downstream integration. A validation-feedback loop keeps the schema strict while giving the model a precise opportunity to repair recoverable extraction errors.

14. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. A senior engineer asks Claude to replace a legacy permissions layer across API handlers, CLI tools, shared middleware, tests, and documentation. The team has not chosen between adapting existing interfaces or introducing a new authorization boundary, and incorrect sequencing could break unrelated workflows.
What should you have Claude do first?
A. Begin direct execution on the highest-traffic package, then let failing tests reveal additional files that require changes.

B. Start a fresh session for each package and avoid sharing findings, preventing earlier assumptions from influencing later edits.
C. Ask Claude to edit every file matching the legacy permission imports, then manually review the diff afterward.
D. Use plan mode to explore dependencies, compare viable migration strategies, and produce an implementation plan before modifying files.
Answer: D
Explanation:
Plan mode is the appropriate first step when a task involves large-scale changes, multiple plausible approaches, architectural decisions, or coordinated multi-file modifications. In practice, it lets Claude inspect the codebase, identify dependencies, compare strategies, and present a reviewable implementation plan before making changes.
The key tradeoff is between *safe upfront design* and faster execution. Direct execution works well for simple, well-scoped edits, but using it for a permissions migration risks discovering design conflicts only after files have already been changed.
Mechanical import editing and test-failure-driven discovery are anti-patterns for architectural migrations because they treat a semantic design change like a simple refactor. Isolating each package in separate sessions without shared findings also undermines consistency across the migration.
Learn more about Claude Code workflows in the Claude Code Overview and related CLI usage patterns in Claude Code CLI.

15. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. Your team added a CI step that reviews code produced by an internal Claude-powered scaffolding workflow. The review job currently resumes the same named Claude session that generated the branch, then asks it to critique the resulting pull request. Human reviewers report that the automated review misses subtle design issues and often defends choices made during generation.
What change would most improve review quality?
A. Ask the generating session to perform three consecutive review passes and report only issues found twice.
B. Resume the generation session after tests complete, then provide failing logs before requesting final review feedback.
C. Run the review in a separate Claude Code invocation with the PR diff, repository context, and review criteria only.
D. Keep the existing session but add instructions requiring Claude to challenge its prior assumptions before approving changes.
Answer: C
Explanation:
Independent review context is important when Claude reviews code that was generated by Claude. A session that already contains the implementation reasoning may be less likely to question those decisions, so a separate Claude Code invocation should receive the PR diff, repository context, and explicit review criteria without the generator's prior chain of work.

The underlying principle is *session context isolation*: generation and review are different roles, and the review role benefits from not inheriting the assumptions, tradeoffs, and justifications from the generation role. This is especially relevant in CI/CD, where automated feedback must be trusted by developers and should not simply validate the implementation path already taken.
Adding skeptical instructions to the same session is weaker because it preserves the original context. Running repeated passes in the same session is also an anti-pattern because it creates apparent rigor without independent judgment, and consensus filtering can hide real but subtle defects. Supplying test logs to the generation session helps with fixing failures, but it does not make the review independent.
Learn more about using Claude Code in automated workflows at Claude Code CLI and broader Claude Code capabilities at Claude Code Overview.

16. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. An engineer asks the agent to investigate a production stack trace from an unfamiliar legacy service. The trace contains the message "InvalidInventoryTransition" and references a helper named validateTransition, but the repository has hundreds of source files and inconsistent directory naming.
What should the agent do first to find the relevant implementation and usage sites efficiently?
A. Use Grep to search file contents for the error string and function names, then Read the matching files.
B. Use Bash to run ad hoc recursive shell commands, then paste the raw terminal output into context.
C. Use Read to load the entire module tree upfront, then ask the agent to infer all relevant references.
D. Use Glob to list likely source files by extension, then Read each candidate file until the error appears.
Answer: A
Explanation:
Grep is the right first tool when the agent needs to search file contents for known strings such as error messages, function names, import statements, or identifiers. In practice, this lets the agent quickly identify candidate implementation and usage sites, then use Read only on the files that matter.
The underlying principle is progressive codebase exploration: start with a narrow content search, then read the smallest useful set of files to understand control flow and dependencies. This preserves context window capacity and avoids attention dilution in large repositories.
Glob is useful for matching file paths, such as **/*.test.tsx, but it does not inspect file contents. Loading broad directory trees with Read is an anti-pattern because it adds irrelevant material to context before the agent knows what matters. Using Bash for ad hoc recursive searches can work in some environments, but raw terminal output is often noisy and less controlled than the purpose-built code search tool.
For more on Claude Code and built-in development workflows, see Claude Code Overview.

17. Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to backend systems through MCP tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate. During evaluation, the coordinator first verifies the customer and retrieves two recent orders, then delegates a billing-dispute investigation. The delegated investigator often asks for the order status

again or analyzes the wrong order when the customer mentioned several purchases. You want the coordinator to preserve accuracy without giving the investigator all support tools.
What should you change?
A. Have the investigator rerun get_customer and lookup_order for every dispute before assessing refund eligibility or billing adjustments.
B. Update coordinator instructions to tell subagents they should remember all facts discovered earlier in the case.
C. Rely on the parent conversation history because Task-spawned investigators can inspect earlier tool results when needed.
D. Include verified customer identity, relevant order records, prior findings, and exact investigation goal in each subagent prompt.
Answer: D
Explanation:
Correct approach: When a coordinator delegates to a subagent, it should provide the relevant case state directly in the subagent invocation. In this situation, the billing investigator needs the verified customer identity, the specific order records, prior tool findings, and the precise investigation goal so it can reason over the correct case without requiring broad backend access.
Underlying principle: Subagents operate with isolated context. They do not automatically inherit the coordinator's conversation history, prior MCP tool results, or hidden memory between invocations, so reliable orchestration requires explicit context passing.
Why the distractors fail: Relying on parent conversation history or telling subagents to remember earlier facts assumes shared context that is not available. Having every subagent rerun lookup tools may appear to recover missing information, but it increases latency, duplicates work, expands tool access, and can still omit coordinator-specific findings or customer statements.
For further study, review subagent orchestration patterns in the Agent SDK documentation and tool interaction fundamentals in Tool Use.

18. Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to backend systems through MCP tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate. In a compliance review, you find several transcripts where the agent called <code>process_refund</code> immediately after a customer claimed an order was damaged. In those cases, <code>lookup_order</code> later showed the order was past the return window or the refund amount differed from the customer's estimate. The current system prompt says to verify customers and check order eligibility before refunds.
What change best reduces this risk?
A. Add a prerequisite gate that rejects process_refund until verified customer, eligible order, and computed refund amount are present in state.
B. Add few-shot examples where the assistant calls get_customer and lookup_order before refunds, including damaged-order and late-return cases.
C. Strengthen the system prompt to require checking customer identity and order eligibility before any refund-related tool call.
D. Add a text parser that permits process_refund only after assistant messages contain verified, eligible,

and refund amount.
Answer: A
Explanation:
Deterministic workflow enforcement is the right approach when a tool action depends on prior steps, especially for financial operations such as refunds. A prerequisite gate can inspect structured state, such as verified customer identity, order eligibility, and computed refund amount, before allowing process_refund to execute.
The underlying architectural tradeoff is between *guidance* and *enforcement*. System prompts and few-shot examples can improve the model's tendency to follow a workflow, but they cannot guarantee ordering when the model is choosing tools autonomously. For business-critical prerequisites, prompt guidance should be backed by programmatic checks.
Using a natural-language parser over assistant text is also unreliable because generated wording is not a source of truth. The safer design is to base decisions on structured tool outputs and explicit state transitions. Learn more about tool orchestration in Tool Use and agent workflow patterns in Agent SDK.

19. Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. Your internal agent is generating repository-specific data adapter classes from interface definitions. In pilots, the first drafts compile, but later reviews find unhandled null fields, incorrect retry behavior, and slow processing on large fixtures. Engineers can state expected outcomes and edge cases before implementation, but prose-only instructions have produced uneven results.
Which workflow should you add?
A. Run the generated code once on a representative happy-path input, then approve if the output matches.
B. Create tests for expected behavior, edge cases, and performance constraints first, then iterate by sending Claude failing results.
C. Let Claude implement the adapter, then rely on manual code review to catch missed cases after generation.
D. Give Claude a longer prose specification and ask it to reason carefully before producing the complete implementation.
Answer: B
Explanation:
Test-driven iteration is the best fit when engineers can define expected behavior, edge cases, and performance constraints before implementation. Creating tests first gives Claude concrete targets, and feeding back failing test output creates a focused refinement loop rather than relying on subjective review comments.
The underlying principle is to convert requirements into *observable acceptance criteria*. Claude can then iterate against specific failures, such as a null field assertion, a retry count mismatch, or a performance threshold violation, instead of guessing what “make it robust” means.
Longer prose specifications can help, but they do not replace executable feedback. Manual review is useful as a safeguard, but it is slower and less systematic than tests for repeated generation workflows. A single happy-path run is especially weak because it ignores the very categories where the pilot is

failing.
For related guidance on improving outputs through examples, constraints, and iterative prompting, see Prompt Engineering. For development workflow context with Claude Code, see Claude Code Overview.

20. Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. During a run, the document-analysis subagent asks a source retrieval MCP tool for the full text of a licensed market report. The backend rejects the request because the license permits summaries and short excerpts only. Today the tool returns “access denied,” causing the coordinator to retry with paraphrased requests and then mark the source as unavailable. What change would best let the agents handle this case correctly?
A. Return a successful empty result set so the synthesis agent can continue without exposing licensing details.
B. Return isError true with errorCategory business, isRetryable false, and guidance to request summaries or short excerpts instead.
C. Report a permission error and instruct the coordinator to escalate to infrastructure owners for immediate credential repair.
D. Classify the rejection as transient and retry with backoff using progressively narrower full-text retrieval queries.
Answer: B
Explanation:
Structured MCP errors should give agents enough information to decide whether to retry, revise the request, explain a limitation, or escalate. In this case, the backend is enforcing a licensing constraint: full-text retrieval is not allowed, but summaries and short excerpts are allowed. Returning isError: true with errorCategory: "business", isRetryable: false, and a human-readable explanation lets the coordinator avoid useless retries and choose an allowed retrieval strategy.
The key principle is distinguishing *business rule violations* from transient failures and permission problems. Transient retry logic is appropriate for timeouts or temporary service unavailability, but it cannot overcome a stable contractual limitation. Returning an empty successful result is an anti-pattern because it hides the difference between “no relevant source exists” and “a relevant source exists but cannot be accessed in that form.” Treating the case as credential repair also misroutes recovery, since the issue is not missing authorization to perform an otherwise permitted action.
Learn more about MCP tool behavior in MCP Tools and Claude tool error handling patterns in Tool Use.

21. Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to backend systems through MCP tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate. A customer starts a chat with: "I don't want to go through a bot. Put me through to a person about order 8421." The message includes enough details that your normal automation would probably resolve the delayed-shipment issue in one or two tool calls. You are tuning the escalation policy and want to avoid both unnecessary deflection and unnecessary automation.

What should the agent do next?

A. Acknowledge the request, run lookup_order to confirm eligibility, then offer the replacement before considering escalation.

B. Use sentiment analysis on the message, escalating only when the customer appears highly frustrated or hostile.

C. Ask the customer to explain why human help is needed, then continue autonomously if the reason seems resolvable.

D. Call escalate_to_human with the order context and brief summary, without attempting autonomous lookup or replacement processing first.

Answer: D

Explanation:

Explicit customer preference is a hard escalation signal. When a customer directly asks for a human agent, the support agent should use escalate_to_human and provide a concise handoff rather than first attempting tool-based resolution.

The tradeoff is between maximizing first-contact resolution and maintaining trust. If the customer only expresses frustration but does not request a person, it can be appropriate to acknowledge the frustration and offer help for a straightforward issue. When the customer clearly asks for a human, continuing automation becomes a deflection pattern.

Using sentiment as the deciding factor is an anti-pattern because frustration level does not reliably indicate case complexity or customer preference. Asking the customer to justify the request also adds unnecessary friction and can worsen the experience.

For implementation guidance on tool-based agent flows and tool result handling, see Tool Use and Agent SDK.

22. Scenario: Structured Data Extraction You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems. Your QA prompt reviews a 90-page vendor onboarding packet and its extracted records in one call. It catches detailed field errors in the first few forms, but misses mismatched tax IDs in middle attachments and contradicts itself about required signatures.

Which review architecture would best improve reliability?

A. Run three full-packet reviews and accept only issues reported by at least two reviewers.

B. Ask the same extraction instance to critique its own output using extended thinking before returning final records.

C. Increase max_tokens and add stronger instructions to be exhaustive while reviewing the entire packet once.

D. Run focused review passes on each packet section, then run a separate consistency pass across cross-section fields.

Answer: D

Explanation:

Focused multi-pass review works because it separates local validation from global consistency analysis. A section-level pass can inspect each form or attachment deeply, while a later cross-section pass can compare fields such as tax IDs, addresses, signatories, and dates across the entire packet.

The underlying tradeoff is *scope versus attention quality*. Large, mixed-objective reviews often produce

uneven coverage, especially when important evidence appears in the middle of long inputs. Splitting the work into narrower passes gives Claude a clearer objective for each call and reduces contradictory findings.
Self-review by the same instance is limited because the model may preserve assumptions from its original extraction. Increasing max_tokens affects available response length, not whether the model attends evenly to every part of a long packet. Majority voting across broad reviews is also an anti-pattern when it filters out valid findings that appear inconsistently rather than fixing the review architecture.
For related guidance on decomposing prompts and improving review reliability, see Prompt Engineering and the Anthropic Cookbook.

23. Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. Your coordinator's document-analysis subagent depends on an internal library-catalog MCP server. It works in your local Claude Code session, but two teammates who cloned the repository cannot access the catalog tools, so their runs fall back to weaker web searches. Separately, one researcher is testing an experimental notes server that should not affect the team.
What should you change?
A. Document per-user claude mcp add commands in README, and ask each researcher to recreate both servers locally.
B. Place both servers in each user's ~/.claude.json, and rely on onboarding scripts to synchronize changes across machines.
C. Combine the catalog and notes integrations into one MCP server, and expose only role-specific tools to subagents.
D. Move the required catalog server into project-scoped .mcp.json, and keep the experimental notes server in user-scoped ~/.claude.json.
Answer: D
Explanation:
Shared MCP configuration should be placed at the project level when the tools are required for a team workflow. A project-scoped .mcp.json travels with the repository, which makes the coordinator and subagents see the same required MCP server after teammates clone or pull the project.
User-scoped configuration, such as ~/.claude.json, is the right place for personal or experimental MCP servers. This prevents one researcher's prototype integration from changing the tool environment for everyone else.
Asking every teammate to recreate shared servers manually is a reliability anti-pattern because configuration drift will produce inconsistent agent behavior. Combining unrelated required and experimental integrations does not address scope, and role-specific tool exposure is a separate design concern from how MCP servers are distributed across a team.
The underlying principle is to match *configuration scope* to the intended audience: project scope for shared team capabilities, user scope for individual tools. Learn more from the Claude Code Overview and the MCP Overview.

24. Scenario: Developer Productivity with Claude You are building developer productivity tools using the

Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses built-in tools (Read, Write, Bash, Grep, Glob) and integrates with MCP servers. The same agent now handles two request types. Dependency license reviews follow a stable rubric with known checks, while production bug investigations in unfamiliar services vary based on discovered call paths, logs, and failing tests. The current single-pass prompt misses checklist items during reviews and wastes effort during investigations. Which decomposition strategy is most appropriate?

A. Use the same fixed sequential checklist for both workflows, requiring every investigation to complete every predefined step.

B. Use dynamic task planning for every workflow, allowing the agent to redefine review criteria as it discovers code patterns.

C. Use a fixed prompt chain for stable review rubrics, and let the agent adapt subtasks during open-ended investigations.

D. Use one large prompt for both request types, asking the agent to reason carefully before using any tools.

Answer: C

Explanation:

Correct approach: Match the decomposition pattern to the workflow shape. A stable dependency license review has known review dimensions, so a fixed prompt chain gives each pass a focused objective and reduces missed checklist items. A production bug investigation in an unfamiliar service is open-ended, so the agent should use adaptive decomposition, creating new subtasks based on discovered call paths, logs, and failing tests.

Architectural principle: Reliable agent design is not about always using the most flexible pattern. It is about choosing the least complex structure that preserves quality: deterministic chains for predictable multi-step work, adaptive plans for exploratory work where intermediate findings change the next best step.

Why the other approaches fail: A single large prompt is an attention dilution anti-pattern because it asks the model to juggle all constraints at once. Applying the same fixed checklist to every investigation wastes effort and can miss emergent leads. Making every workflow dynamically planned adds unnecessary variability to stable reviews and may weaken consistency when the criteria should remain fixed.

For related guidance on agentic workflows and decomposition, see the Agent SDK documentation and Prompt Engineering.

25. Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to backend systems through MCP tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate. During rollout, the agent frequently spends its first several turns probing the support knowledge server with broad queries such as “what policies exist for billing disputes?” and “which return exceptions are available?” These exploratory calls consume context, sometimes miss relevant policy documents, and delay straightforward resolutions before any customer-specific action occurs.

What design change would best improve the agent’s visibility into available support information while preserving tools for customer-specific actions?

A. Expose the support policy index and knowledge-base catalog as MCP resources, while keeping tools for selected record actions.
B. Build keyword rules that map phrases like refund, late, or angry to predefined policy documents before tool use.
C. Add a mandatory startup tool call that lists every policy document before the agent handles each customer request.
D. Copy the full return, billing, and escalation knowledge base into the system prompt for every agent session.
Answer: A
Explanation:
MCP resources are designed for exposing read-oriented information such as content catalogs, documentation hierarchies, policy indexes, or database schemas. In this scenario, a policy catalog resource lets the agent see what support information exists without spending several tool calls discovering it, while MCP tools remain reserved for actions such as get_customer, lookup_order, process_refund, and escalate_to_human.
The underlying architectural principle is to separate *visibility into available content* from *model-initiated actions*. Resources help reduce unnecessary exploratory tool calls, preserve context budget, and improve the agent's ability to choose relevant information for the customer's issue.
A mandatory startup listing tool still forces an action on every request and adds latency. Copying the full knowledge base into the system prompt creates context bloat and makes policy updates less manageable. Keyword routing is an anti-pattern for ambiguous support language because it relies on brittle surface matches rather than structured visibility into the support content available.
Learn more about MCP primitives at MCP Overview and the resource pattern at MCP Resources.

26. Scenario: Customer Support Resolution Agent You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to backend systems through MCP tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate. A customer writes: "I was charged twice, my return label never arrived, and I cannot sign in to check the order." The coordinator currently launches billing, returns, and account subagents with the same prompt: "Investigate this customer issue and recommend a resolution." Logs show each subagent calls overlapping tools for the same order, repeats the same facts, and sometimes recommends conflicting next steps.
What orchestration change would most effectively improve reliability and efficiency?
A. Route every request through returns, billing, and account subagents sequentially, regardless of the customer's stated concerns.
B. Send the full customer transcript to every subagent, then use majority agreement to choose the resolution.
C. Assign each subagent a distinct issue boundary and shared case facts, then synthesize their non-overlapping findings centrally.
D. Allow subagents to message each other directly so they can divide remaining work without coordinator involvement.
Answer: C
Explanation:

Thank you for Trying our Free Sample Questions

But We Recommend try our Premium Exam Material (Full Premium PDF) dumps in PDF Format to certain your Guaranteed success in First Attempt Only.

VISIT US NOW TO DOWNLOAD FULL PDF INSTANTLY

https://www.clearcatnet.com/Papers



Send us your request/inquiry at clearcat.net@gmail.com or connect us for Live Support any time for **any certification exam dumps pdf** Or for **most asked Interview Q&A PDFs** to ensure your success in first try!!

Get any exam latest real exam questions PDF Now-

- **Visit us** - www.clearcatnet.com
- **Mail us**- clearcat.net@gmail.com
- **Live Support**- https://t.me/clearcatnet

YouTube.com /CLEARCATNET

t.Me /CLEARCATNET

IT Certifications Exam – Best Practice Q&A

Clearcat.net
Be Certified Now!

97.5% Success Rate

Get Any Full Premium PDF Now

WWW.CLEARCATNET.COM



Mail us- clearcat.net@gmail.com

Live Chat t.Me/CLEARCATNET

Watch us- YouTube/CLEARCATNET



Microsoft Azure aws Google CISCO CompTIA Get IT Certified

Latest Real Exam Q&A, FIRST ATTEMPT PASS